

```
import pandas as pd
data=pd.read_csv('ev.csv')
```

```
display(data.head())
```

```
data.info()
```

	Trip_ID	Vehicle_ID	Driver_ID	Manufacturer	Model	Registration_No	Batt
0	TRIP-100001	EV-1005	DRV-5005	Hyundai	Kona EV	REG-75PW-9155	
1	TRIP-100002	EV-1035	DRV-5035	Kia	EV9	REG-36HZ-1495	
2	TRIP-100003	EV-1020	DRV-5020	BYD	Atto 3	REG-53MS-1284	
3	TRIP-100004	EV-1001	DRV-5001	Hyundai	Kona EV	REG-85BE-3789	
4	TRIP-100005	EV-1018	DRV-5018	Hyundai	Kona EV	REG-77ZN-8883	

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 10000 entries, 0 to 9999
```

```
Data columns (total 19 columns):
```

#	Column	Non-Null Count	Dtype
0	Trip_ID	10000 non-null	object
1	Vehicle_ID	10000 non-null	object
2	Driver_ID	10000 non-null	object
3	Manufacturer	10000 non-null	object
4	Model	10000 non-null	object
5	Registration_No	10000 non-null	object
6	Battery_Capacity_kWh	10000 non-null	int64
7	Current_Battery_%	10000 non-null	int64
8	Battery_Remaining_%	10000 non-null	int64
9	Expected_Range_km	10000 non-null	int64
10	Status	10000 non-null	object
11	Date_Time	10000 non-null	object
12	Start_Time	10000 non-null	object
13	Distance_km	10000 non-null	float64
14	Avg_Speed_kmph	10000 non-null	float64
15	Revenue	10000 non-null	float64
16	Electricity_Cost	10000 non-null	float64
17	Maintenance_Cost	10000 non-null	float64
18	Overspeed_Count	10000 non-null	int64

```
dtypes: float64(5), int64(5), object(9)
```

```
memory usage: 1.4+ MB
```

```

running_count = data[data['Status'] == 'Running'].shape[0]
print(f"Vehicles Running: {running_count}")

garage_count = data[data['Status'] == 'In Garage'].shape[0]
print(f"Vehicles In Garage: {garage_count}")

charging_count = data[data['Status'] == 'Charging'].shape[0]
print(f"Vehicles Charging: {charging_count}")

```

```

Vehicles Running: 3322
Vehicles In Garage: 0
Vehicles Charging: 3445

```

```

unique_manufacturers = data['Manufacturer'].unique()
num_manufacturers = len(unique_manufacturers)
print(f"Unique EV Manufacturers: {unique_manufacturers}")
print(f"Number of Manufacturers: {num_manufacturers}")

if num_manufacturers >= 4:
    print("There are at least 4 EV manufacturers included in the dataset.")
else:
    print("There are fewer than 4 EV manufacturers in the dataset.")

```

```

Unique EV Manufacturers: ['Hyundai' 'Kia' 'BYD' 'Tata' 'Tesla']
Number of Manufacturers: 5
There are at least 4 EV manufacturers included in the dataset.

```

```

manufacturer_counts = data['Manufacturer'].value_counts()
print("Manufacturer-wise Vehicle Count:")
print(manufacturer_counts)

```

```

Manufacturer-wise Vehicle Count:
Manufacturer
Tesla      3194
BYD        2196
Hyundai    2033
Kia        1573
Tata       1004
Name: count, dtype: int64

```

✓ Query 7: Month-wise revenue generation for each car

```

data['Start_Time'] = pd.to_datetime(data['Start_Time'])
data['Revenue'] = pd.to_numeric(data['Revenue'])

data['YearMonth'] = data['Start_Time'].dt.to_period('M')

month_wise_revenue = data.groupby(['Vehicle_ID', 'YearMonth'])['Revenue'].sum()

print("Month-wise Revenue Generation for Each Car:")
for vehicle_id in month_wise_revenue['Vehicle_ID'].unique():
    print(f"\nVehicle: {vehicle_id}")

```

```
vehicle_revenue = month_wise_revenue[month_wise_revenue['Vehicle_ID'] ==  
for index, row in vehicle_revenue.iterrows():  
    print(f"    {row['YearMonth'].strftime('%B %Y')} → ₹{row['Revenue']:},}
```

July 2026 → ₹12,767.93

✓ Visualization: Month-wise Revenue Generation for Each Car

```
import matplotlib.pyplot as plt
import seaborn as sns

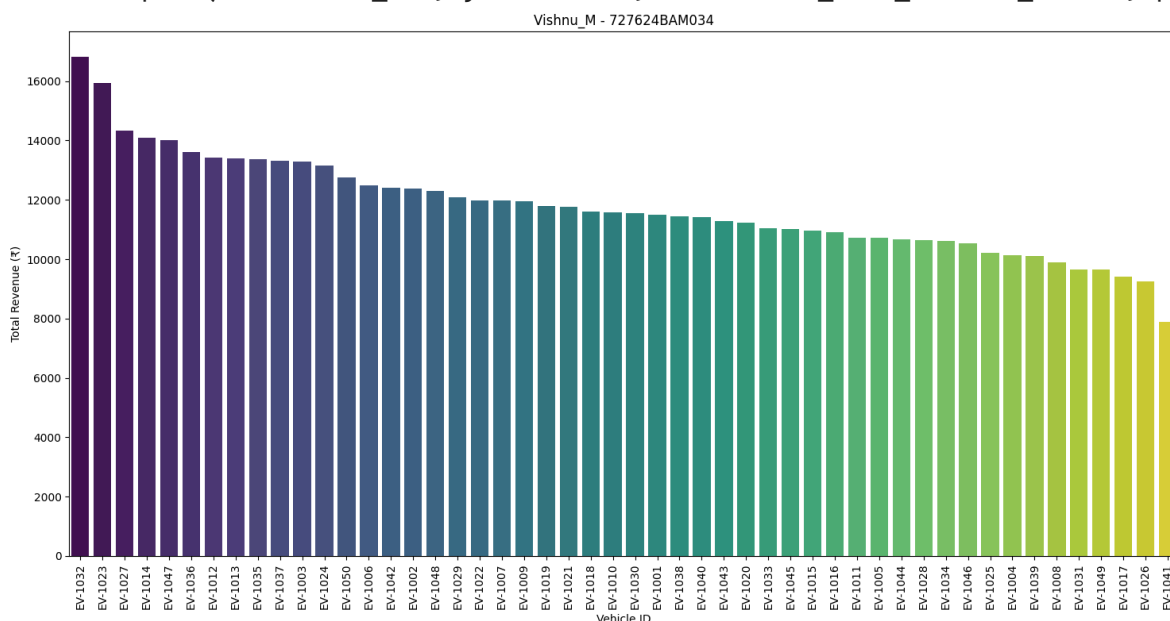
month_wise_revenue_sorted = month_wise_revenue.sort_values(by='Revenue', ascending=False)

plt.figure(figsize=(15, 8))
sns.barplot(x='Vehicle_ID', y='Revenue', data=month_wise_revenue_sorted, palette='magma')
plt.title('Vishnu_M - 727624BAM034')
plt.xlabel('Vehicle ID')
plt.ylabel('Total Revenue (₹)')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```

/tmp/ipykernel_1208/1145889337.py:7: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in a future version. Please use `color` or `colorbar` instead.

```
sns.barplot(x='Vehicle_ID', y='Revenue', data=month_wise_revenue_sorted, palette='magma')
```



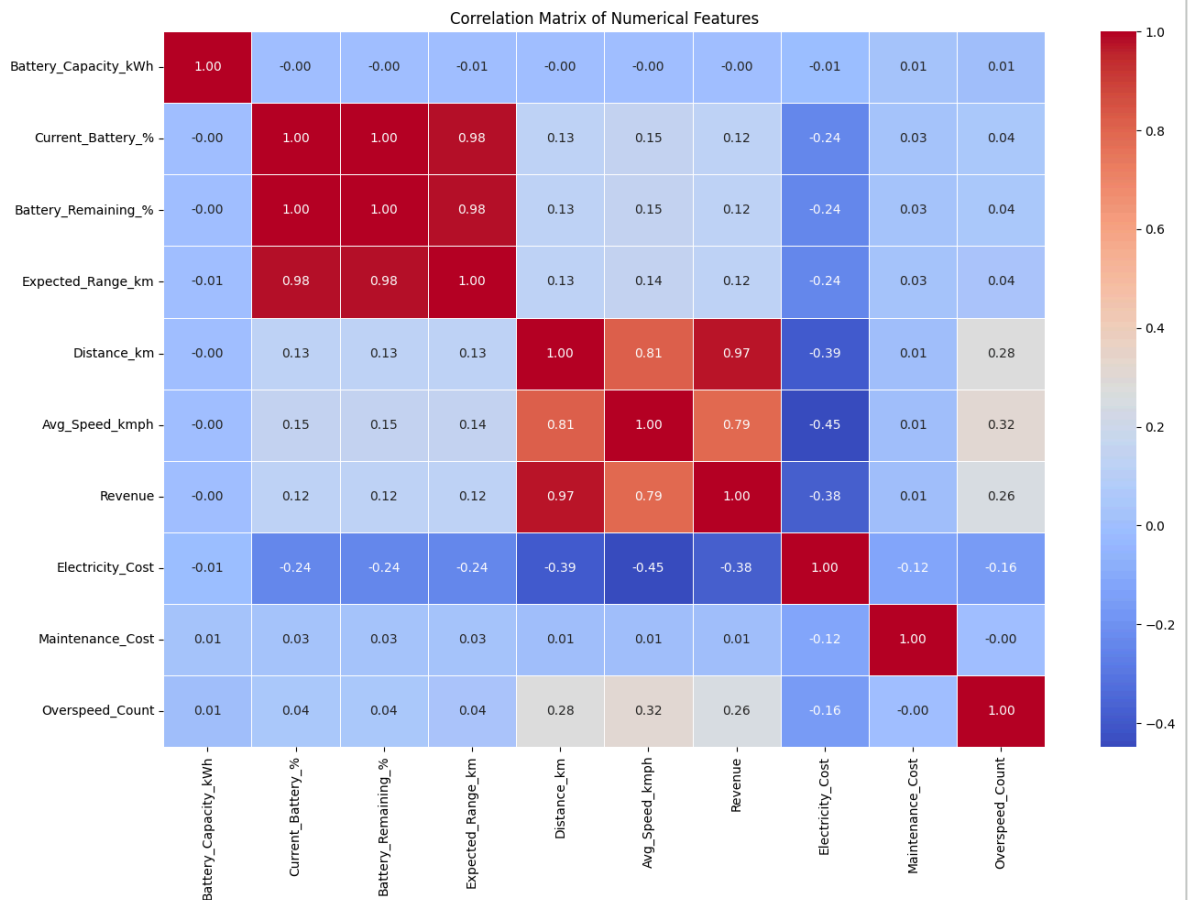
✓ Correlation Heatmap of Numerical Features

```
import matplotlib.pyplot as plt
import seaborn as sns

numerical_data = data.select_dtypes(include=['int64', 'float64'])

correlation_matrix = numerical_data.corr()
```

```
plt.figure(figsize=(14, 10))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f", line
plt.title('Correlation Matrix of Numerical Features')
plt.tight_layout()
plt.show()
```



✓ Query 8: Month-wise maintenance cost for each car

```
data['Maintenance_Cost'] = pd.to_numeric(data['Maintenance_Cost'])

month_wise_maintenance = data.groupby(['Vehicle_ID', 'YearMonth'])['Maintenance_Cost']

print("Month-wise Maintenance Cost for Each Car:")
for vehicle_id in month_wise_maintenance['Vehicle_ID'].unique():
    print(f"\nVehicle: {vehicle_id}")
    vehicle_maintenance = month_wise_maintenance[month_wise_maintenance['Vehicle_ID'] == vehicle_id]
    for index, row in vehicle_maintenance.iterrows():
        print(f"    {row['YearMonth'].strftime('%B %Y')} → ₹{row['Maintenance_Cost']}
```

```

Vehicle: EV-1034
  July 2026 → ₹1,055.43

Vehicle: EV-1035
  July 2026 → ₹758.6899999999999

Vehicle: EV-1036
  July 2026 → ₹998.77

Vehicle: EV-1037
  July 2026 → ₹794.88

Vehicle: EV-1038
  July 2026 → ₹893.87

Vehicle: EV-1039
  July 2026 → ₹663.45

Vehicle: EV-1040
  July 2026 → ₹1,149.03

Vehicle: EV-1041
  July 2026 → ₹785.86

Vehicle: EV-1042
  July 2026 → ₹909.85

Vehicle: EV-1043
  July 2026 → ₹264.21

Vehicle: EV-1044
  July 2026 → ₹524.89

Vehicle: EV-1045
  July 2026 → ₹505.32

Vehicle: EV-1046
  July 2026 → ₹823.5500000000001

Vehicle: EV-1047
  July 2026 → ₹605.52

Vehicle: EV-1048
  July 2026 → ₹850.0

Vehicle: EV-1049
  July 2026 → ₹498.1

Vehicle: EV-1050
  July 2026 → ₹1,033.53

```

✓ Visualization: Month-wise Maintenance Cost for Each Car

```

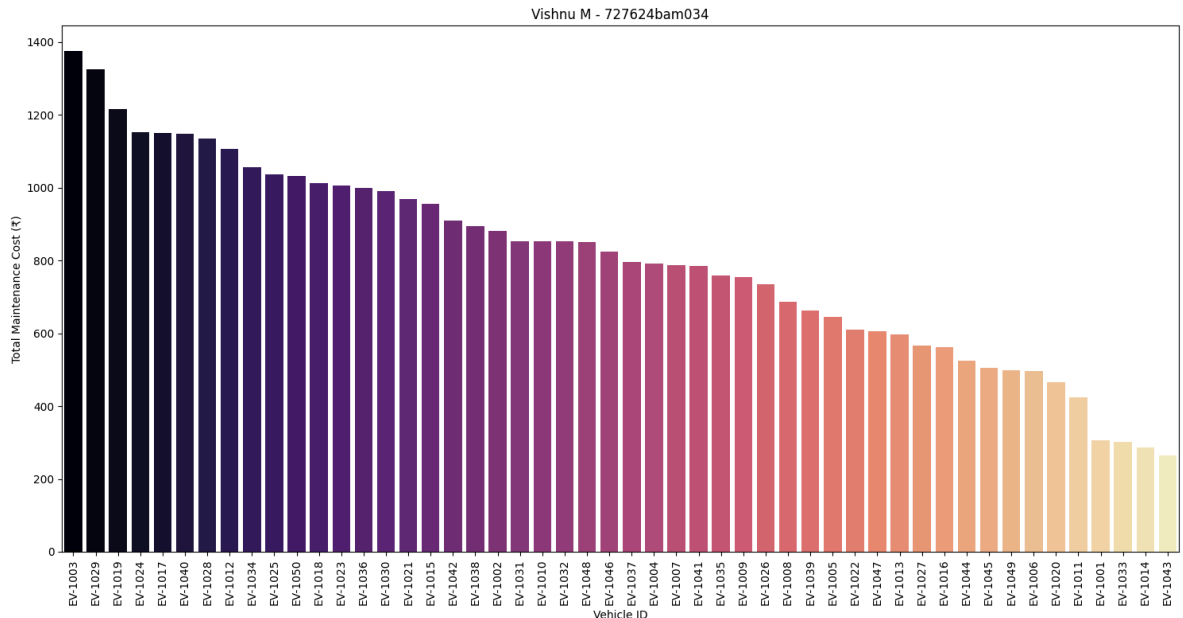
import matplotlib.pyplot as plt
import seaborn as sns

month_wise_maintenance_sorted = month_wise_maintenance.sort_values(by='Mai

plt.figure(figsize=(15, 8))

```

```
sns.barplot(x='Vehicle_ID', y='Maintenance_Cost', data=month_wise_maintenai
plt.title('Vishnu M - 727624bam034')
plt.xlabel('Vehicle ID')
plt.ylabel('Total Maintenance Cost (₹)')
plt.xticks(rotation=90)
plt.tight_layout()
plt.show()
```



✓ Query 9: One car is used by one driver only

```
driver_per_vehicle = data.groupby('Vehicle_ID')['Driver_ID'].nunique()

if (driver_per_vehicle == 1).all():
    print("Assumption holds: Each vehicle is used by one driver only.")
else:
    print("Assumption does not hold: Some vehicles are used by multiple driv
    print("Vehicles with multiple drivers:\n", driver_per_vehicle[driver_per

print("\nExample assignments:")
display(data[['Vehicle_ID', 'Driver_ID']].drop_duplicates().head())
```

Assumption holds: Each vehicle is used by one driver only.

Example assignments:

	Vehicle_ID	Driver_ID	
0	EV-1005	DRV-5005	
1	EV-1035	DRV-5035	
2	EV-1020	DRV-5020	
3	EV-1001	DRV-5001	
4	EV-1018	DRV-5018	

✓ Query 10: Which driver makes maximum and minimum revenue

```
driver_revenue = data.groupby('Driver_ID')['Revenue'].sum()

max_revenue_driver = driver_revenue.idxmax()
max_revenue_amount = driver_revenue.max()

min_revenue_driver = driver_revenue.idxmin()
min_revenue_amount = driver_revenue.min()

print("Maximum Revenue:")
print(f"Driver {max_revenue_driver} → ₹{max_revenue_amount:,.0f}")

print("\nMinimum Revenue:")
print(f"Driver {min_revenue_driver} → ₹{min_revenue_amount:,.0f}")
```

Maximum Revenue:
Driver DRV-5032 → ₹16,826

Minimum Revenue:
Driver DRV-5041 → ₹7,881

✓ Query 11: Maximum overspeed violation by which driver

```
data['Overspeed_Count'] = pd.to_numeric(data['Overspeed_Count'])

driver_overspeed = data.groupby('Driver_ID')['Overspeed_Count'].sum()

max_overspeed_driver = driver_overspeed.idxmax()
max_overspeed_count = driver_overspeed.max()

print("Maximum Overspeed Violation:")
print(f"Driver {max_overspeed_driver} → {int(max_overspeed_count)} violation")
```

Maximum Overspeed Violation:
Driver DRV-5047 → 35 violations

✓ Query 12: Date and time column

```
data['Date_Time'] = pd.to_datetime(data['Date_Time'])

print("Sample Date and Time values from 'Date_Time' column:")
display(data['Date_Time'].head().dt.strftime('%Y-%m-%d %H:%M:%S'))

print("\nSample Date and Time values from 'Start_Time' column:")
display(data['Start_Time'].head().dt.strftime('%Y-%m-%d %H:%M:%S'))
```


Sample Date and Time values from 'Date_Time' column:

	Date_Time
0	2026-07-15 18:45:24
1	2026-08-04 18:01:15
2	2026-03-24 11:00:40
3	2026-01-22 00:50:22
4	2026-02-06 09:27:16

dtype: object

Sample Date and Time values from 'Start_Time' column:

	Start_Time
0	2026-07-07 14:19:00
1	2026-07-07 07:34:00
2	2026-07-07 02:25:00
3	2026-07-07 20:09:00
4	2026-07-07 09:54:00

dtype: object

✓ Query 13: Present battery percentage and expected travel distance

```
print("Sample Battery Percentage and Expected Travel Distance:")
display(data[['Vehicle_ID', 'Current_Battery_%', 'Expected_Range_km']].head(5))
```

Sample Battery Percentage and Expected Travel Distance:

	Vehicle_ID	Current_Battery_%	Expected_Range_km	
0	EV-1005	35	115	
1	EV-1035	78	301	
2	EV-1020	57	186	
3	EV-1001	47	177	
4	EV-1018	83	348	

✓ Query 14: Use the dataset in an app

This dataset is well-suited for an application displaying EV fleet management metrics. Here's how it can be used:

- **Running, Charging, Garage Vehicles:** The `Status` column can be used to filter and display counts of vehicles in these states.
- **Manufacturer Statistics:** The `Manufacturer` column provides data for grouping and displaying vehicle counts per manufacturer.
- **Monthly Revenue & Maintenance Cost:** By processing the `Start_Time`, `Revenue_Generated`, and `Maintenance_Cost` columns, an app can show monthly financial performance for individual vehicles or the entire fleet.
- **Driver Performance:** `Driver_ID`, `Revenue_Generated`, and `Overspeed_Violations` can be combined to assess and display driver performance metrics.
- **Overspeed Reports:** The `Overspeed_Violations` column can be used to generate reports on drivers with high violation counts.
- **Battery Level & Expected Range:** The `Battery_Level` and `Expected_Range_km` columns provide real-time (or last-recorded) information on vehicle battery status and potential travel distance, which is crucial for operational planning.